

Crypto Toolkit Project

Cryptography

Mahmoud Azmy

Mohmed Adel

Mohmed Abd El Aziz

Course Number: Cryptography

Introduction

The **Crypto Toolkit** is a graphical application designed to provide a comprehensive suite of cryptographic tools, including symmetric and asymmetric encryption, data encoding, and one-way hashing functions. Built using Python and the Tkinter framework, the application serves as a practical implementation of standard security protocols, providing a user-friendly interface for complex mathematical operations.

Algorithm used:

2.1 Symmetric Encryption (AES, DES, 3DES)

Symmetric encryption uses the same secret key for both encryption and decryption. The application implements block ciphers that support various modes of operation (ECB, CBC, CFB, OFB, CTR).

- **AES (Advanced Encryption Standard):** The current global standard (FIPS 197). It operates on 128-bit blocks and supports key lengths of 128, 192, and 256 bits. It is based on a substitution-permutation network.
- **DES (Data Encryption Standard):** A legacy 64-bit block cipher using a 56-bit effective key. It follows a 16-round Feistel network structure. It is included for educational purposes.
- **3DES (Triple DES):** Enhances DES security by applying the DES cipher three times to each block (Encrypt-Decrypt-Encrypt), effectively increasing the key length to 112 or 168 bits.

2.2 Asymmetric Encryption (RSA)

- **RSA (Rivest-Shamir-Adleman):** Unlike symmetric ciphers, RSA uses a public key for encryption and a private key for decryption. Its security relies on the factoring of large prime numbers. This

application supports 1024-bit and 2048-bit keys and uses **OAEP (Optimal Asymmetric Encryption Padding)** with SHA-256 for enhanced security against modern attacks.

2.3 Encoding Techniques

Encoding is used to transform binary data into printable ASCII strings for easy transmission. It does not provide secrecy.

- **Base64:** Converts 3 bytes of data into 4 characters from a 64-character alphabet. Commonly used for transmitting ciphertext in text-based protocols.
- **Hex (Hexadecimal):** Represents each byte as two characters (0-9, a-f). It is the standard for displaying keys and digests.
- **URL Encoding:** Replaces non-ASCII or reserved characters with a % followed by two hex digits to ensure data can be safely passed in a URI.

2.4 Hashing Algorithms

Hashing is a one-way process that creates a fixed-size "fingerprint" of the input data.

- **SHA-256:** Part of the SHA-2 family, producing a 256-bit (32-byte) digest. It is widely used for digital signatures and data integrity.

- **SHA-512:** Similar to SHA-256 but uses 64-bit words and produces a 512-bit digest. It is faster on 64-bit architectures and provides a higher security margin.
- **Salted Hashing:** To prevent rainbow table attacks, a random 16-byte **salt** is generated and prepended to the password before hashing. This ensures that identical passwords result in different hashes.

Implementation Screenshots:

crypto_lib.py

Standard liberals

```

28 # Standard library
29 import os                                # os.urandom() – cryptographic random bytes
30 import base64 as _b64                    # built-in base64 encode/decode
31 import hashlib                            # SHA-256, SHA-512
32 import hmac                              # constant-time compare (for salted verify)
33 import secrets                           # secrets.choice() – secure random picks
34 import string                             # string.ascii_* / digits / punctuation
35 import re                                # regex for pattern detection in strength test
36 from urllib.parse import (               # URL encoding / decoding
37     quote as _url_quote,
38     unquote as _url_unquote,
39 )

```

Section 0 - helper methods

```

74 # -----SECTION 0 – SHARED HELPERS-----
75 # change bytes to int numbers
76 def _bytes_to_int(b: bytes) -> int:
77     return int.from_bytes(b, "big")      # built-in replaces the manual loop
78
79 # change int to bytes
80 def _int_to_bytes(n: int, length: int) -> bytes:
81     """Convert an integer to big-endian bytes of exactly `length` bytes."""
82     return n.to_bytes(length, "big")     # built-in replaces the manual loop
83
84 #xor operation
85 def _xor_bytes(a: bytes, b: bytes) -> bytes:
86     """XOR two equal-length byte strings."""
87     return bytes(x ^ y for x, y in zip(a, b)) # kept as-is – already simple
88
89 # in the whole code we using zero bading
90 def _zero_pad(data: bytes, block_size: int) -> bytes:
91     """Right-pad data with zero bytes to the next multiple of block_size."""
92     remainder = len(data) % block_size
93     if remainder == 0:
94         return data                      # already aligned – nothing to do
95     return data + b'\x00' * (block_size - remainder) # append zero bytes
96
97 # also help in unpadding and decoding sections
98 def _unpad_zeros(data: bytes) -> bytes:
99     """Strip trailing zero bytes that were added by _zero_pad."""
100    return data.rstrip(b'\x00')           # built-in bytes method
101
102 # Add 1 to a big-endian counter block, wrapping on overflow.
103 def _increment_counter(counter: bytes) -> bytes:
104     n = _bytes_to_int(counter) + 1      # convert, increment
105     return _int_to_bytes(n % (1 << (len(counter) * 8)), len(counter)) # wrap

```

Hex encoding

```

122 #-----SECTION 2 - HEX ENCODING-----
123
124 # Uses bytes.hex() and bytes.fromhex().
125 def hex_encode(data: bytes) -> str:
126     """
127     Encode raw bytes to a lowercase hex string.
128
129     Uses the built-in bytes.hex() method which produces two hex chars per byte.
130
131     Parameters
132     -----
133     data : bytes
134
135     Returns
136     -----
137     str - lowercase hex string (2 characters per input byte)
138     """
139     return data.hex()          # e.g. b'\xde\xad' -> 'dead'
140
141 #decoding using bytes library also
142 def hex_decode(s: str) -> bytes:
143     """
144     Decode a hex string back to raw bytes.
145
146     Uses the built-in bytes.fromhex() which validates and converts the string.
147
148     Parameters
149     -----
150     s : str - hex-encoded string (must have even length)
151
152     Returns
153     -----
154     bytes
155     """
156     return bytes.fromhex(s)    # e.g. 'dead' -> b'\xde\xad'

```


URL encoding

```

159 # -----SECTION 3 - URL ENCODING (RFC 3986)-----
160
161 # Uses urllib.parse.quote / unquote.
162 def url_encode(text: str) -> str:
163     """
164     Percent-encode a string per RFC 3986.
165
166     Uses urllib.parse.quote() with safe='' so every non-unreserved character
167     is encoded, including spaces (→ %20) and Arabic / Unicode characters.
168
169     Parameters
170     -----
171     text : str
172
173     Returns
174     -----
175     str - percent-encoded string
176     """
177     return _url_quote(text, safe="") # safe='' means nothing is left unencoded
178
179 #decoding section for url
180 def url_decode(text: str) -> str:
181     """
182     Decode a percent-encoded string back to plain Unicode text.
183
184     Uses urllib.parse.unquote() which handles %XX sequences and UTF-8 decoding.
185     Supports Arabic, emoji, and all Unicode characters.
186
187     Parameters
188     -----
189     text : str - percent-encoded string
190
191     Returns
192     -----
193     str - decoded plain text
194     """
195     return _url_unquote(text) # handles %XX → character, UTF-8 aware
196

```

SHA 256

```

198 #-----SECTION 4 - SHA-256-----
199
200 # Uses hashlib.sha256 - part of Python's standard library.
201 def sha256(data: bytes) -> bytes:
202     """
203     Compute the SHA-256 digest of data.
204
205     Uses Python's built-in hashlib.sha256 (backed by OpenSSL internally).
206
207     Parameters
208     -----
209     data : bytes
210
211     Returns
212     -----
213     bytes - 32-byte (256-bit) digest
214     """
215     return hashlib.sha256(data).digest() # .digest() returns raw bytes
216
217
218 def sha256_hex(data: bytes) -> str:
219     """Return SHA-256 digest as a lowercase hex string."""
220     return hashlib.sha256(data).hexdigest() # .hexdigest() returns lowercase hex
221

```

SHA 512

```

224 #-----SECTION 5 - SHA-512-----
225
226 # Uses hashlib.sha512.
227 def sha512(data: bytes) -> bytes:
228     """
229     Compute the SHA-512 digest of data.
230
231     Uses Python's built-in hashlib.sha512.
232
233     Parameters
234     -----
235     data : bytes
236
237     Returns
238     -----
239     bytes - 64-byte (512-bit) digest
240     """
241     return hashlib.sha512(data).digest() # .digest() returns raw bytes
242
243 # this part to make it able to read
244 def sha512_hex(data: bytes) -> str:
245     """Return SHA-512 digest as a lowercase hex string."""
246     return hashlib.sha512(data).hexdigest() # .hexdigest() returns lowercase hex
247

```

Salted hashing

```

249 #-----SECTION 6 – SALTED HASHING-----
250
251 # SHA-256 with a random 16-byte salt prepended before hashing.
252 _SALT_BYTES = 16 # 128-bit random salt – same size as original
253
254 # the salted hashing
255 def salted_hash(password: str, salt: bytes = None) -> dict:
256     """
257     Hash a password with a random salt using SHA-256.
258
259     Construction: digest = SHA-256( salt || password_utf8 )
260
261     Parameters
262     -----
263     password : str – plain-text password
264     salt      : bytes – optional; generated randomly if not provided
265
266     Returns
267     -----
268     dict with:
269     |   'salt' : hex-encoded salt string
270     |   'hash' : hex-encoded SHA-256 digest string
271     """
272     if salt is None:
273         salt = os.urandom(_SALT_BYTES) # 16 cryptographically random bytes
274     combined = salt + password.encode("utf-8") # prepend salt to password bytes
275     digest = hashlib.sha256(combined).digest() # SHA-256 of salt+password
276     return {
277         "salt": hex_encode(salt), # return hex strings for easy storage
278         "hash": hex_encode(digest),
279     }

```

AES encoding

```

306 #-----SECTION 7 – AES (uses cryptography library)-----
307
308 # Modes supported: ECB, CBC, CFB, OFB, CTR, GCM
309 # Key sizes      : 16 bytes (AES-128), 24 bytes (AES-192), 32 bytes (AES-256)
310 # Padding        : Zero-padding for block modes; stream modes need no padding
311 # Output         : Base64 string for all encrypt() calls
312

```

```

313 #the AES class
314 class AES:
315     """
316     AES encryption / decryption – same API as the original hand-coded version.
317
318     Usage (unchanged from original):
319     | aes = AES(key)
320     | ct = aes.encrypt(plaintext, mode='CBC', iv=my_iv)
321     | pt = aes.decrypt(ct, mode='CBC', iv=my_iv)
322
323     GCM mode returns a dict: {'ciphertext': b64, 'tag': b64, 'iv': b64}
324     All other modes return a Base64 string.
325     """
326
327     BLOCK = 16 # AES block size is always 128 bits = 16 bytes
328
329     #Store the key; validation happens inside each cipher instantiation
330     def __init__(self, key: bytes):
331         _require_crypto() # fail fast if library not installed
332         if len(key) not in (16, 24, 32):
333             raise ValueError("AES key must be 16, 24, or 32 bytes.")
334         self._key = key # store key for later use
335
336 # the main code

```

Modes: for e.g CBC

```

385 # code by mode CBC
386 if mode == 'CBC':
387     data = _zero_pad(plaintext, self.BLOCK)
388     cipher = Cipher(algorithms.AES(self._key), modes.CBC(iv),
389                    backend=default_backend())
390     enc = cipher.encryptor()
391     ct = enc.update(data) + enc.finalize()
392

```

AES decoding

```

422     # the main decoding section
423     def decrypt(self, ciphertext_b64: str, mode: str = 'CBC',
424                 iv: bytes = None, tag: bytes = None, aad: bytes = b'') -> bytes:
425         """
426         Decrypt a Base64-encoded ciphertext and return plaintext bytes.
427
428         Parameters
429         -----
430         ciphertext_b64 : str    - Base64 ciphertext (or raw bytes for GCM)
431         mode           : str
432         iv             : bytes - must match the IV used during encrypt
433         tag            : bytes - GCM authentication tag
434         aad            : bytes - additional authenticated data (GCM only)
435
436         Returns
437         -----
438         bytes - plaintext
439         """
440         mode = mode.upper()
441         ct = base64_decode(ciphertext_b64) if isinstance(ciphertext_b64, str) \
442            | else ciphertext_b64                # accept both str and bytes
443
444         # decoding by mode GCM
445         if mode == 'GCM':
446             if iv is None or tag is None:
447                 raise ValueError("GCM decrypt requires both iv and tag.")
448             aesgcm = AESGCM(self._key)
449             combined = ct + tag                # AESGCM.decrypt expects ct+tag
450             return aesgcm.decrypt(iv, combined, aad or None)
451

```

mode: CBC

```

463     # decoding by mode CBC
464     if mode == 'CBC':
465         cipher = Cipher(algorithms.AES(self._key), modes.CBC(iv),
466                         backend=default_backend())
467         dec = cipher.decryptor()
468         pt = dec.update(ct) + dec.finalize()
469         return _unpad_zeros(pt)

```

DES endoing

```
495 #-----SECTION 8 - DES  (uses cryptography library)-----
496 #
497 # Key size : 8 bytes (64-bit key, 56 effective bits)
498 # Block    : 8 bytes
499 # Modes     : ECB, CBC, CFB, OFB, CTR
500 # Note      : DES is deprecated - provided for educational purposes only.
501
```



modes

```

528         elif mode_str == 'CBC':
529             m = modes.CBC(iv)
530         elif mode_str == 'CFB':
531             m = modes.CFB(iv)           # 8-bit CFB by default for DES
532         elif mode_str == 'OFB':

```

3DES encoding and modes types

```

572 #-----SECTION 9 - TRIPLE-DES (uses cryptography library)-----
573 #
574 # Key sizes : 16 bytes (2-key EDE) or 24 bytes (3-key EDE)
575 # Block      : 8 bytes
576 # Modes      : ECB, CBC, CFB, OFB, CTR
577
578 # 3DES main class
579 class TripleDES:
580     """
581     3DES (Triple-DES) encryption / decryption - same API as the original.
582
583     Usage:
584         tdes = TripleDES(key) # key = 16 or 24 bytes
585         ct  = tdes.encrypt(plaintext, mode='CBC', iv=iv)
586         pt  = tdes.decrypt(ct, mode='CBC', iv=iv)
587     """
588
589     BLOCK = 8 # 3DES block size = 64 bits = 8 bytes
590     # Internal helper:
591     def __init__(self, key: bytes):
592         _require_crypto()
593         if len(key) not in (16, 24):
594             raise ValueError("3DES key must be 16 or 24 bytes.")
595         self._key = key
596     def _make_cipher(self, mode_str: str, iv: bytes = None):
597         """Build a cryptography Cipher object for 3DES."""
598         mode_str = mode_str.upper()
599         if mode_str == 'ECB':
600             m = modes.ECB()
601         elif mode_str == 'CBC':
602             m = modes.CBC(iv)
603         elif mode_str == 'CFB':
604             m = modes.CFB(iv)

```

RSA

```

641 #-----SECTION 10 - RSA (uses cryptography library)-----
642 # Key sizes : 1024 or 2048 bits
643 # Padding : OAEP (encrypt/decrypt) - stronger than PKCS#1 v1.5
644 # Operations : generate, encrypt, decrypt
645 # Key format : hex-encoded for GUI display / paste (same as original)
646
647 # RSA class code
648 class RSA:
649     """
650     RSA public-key cryptography - same public API as the original.
651
652     Usage:
653         rsa = RSA.generate(bits=2048)
654         ct = rsa.encrypt(b"Hello")
655         pt = rsa.decrypt(ct)
656         pub_hex = rsa_pub_hex(rsa) # export for display / storage
657         priv_hex = rsa_priv_hex(rsa) # export for display / storage
658
659     Internal representation uses cryptography library key objects.
660     The ._key (private) and ._pub (public) attributes hold those objects.
661     The ._n, ._e, ._d properties expose the raw integers for the GUI helpers.
662     The ._k property gives the modulus byte length (needed by GUI helpers).
663     """
664     # Internal helper:
665     def __init__(self, n: int, e: int, d: int = None):
666         """
667         Reconstruct an RSA instance from raw integer components.
668         Used by rsa_from_hex() to rebuild from pasted hex fields.
669         """
670         _require_crypto()
671         self.n = n # modulus
672         self.e = e # public exponent
673         self.d = d # private exponent (None = public only)
674         self.k = (n.bit_length() + 7) // 8 # modulus byte length
675

```

encoding

```

772 # encryption main section
773 def encrypt(self, plaintext: bytes) -> str:
774     """
775     Encrypt plaintext with the RSA public key (OAEP padding).
776
777     Parameters
778     -----
779     plaintext : bytes - raw message
780
781     Returns
782     -----
783     str - Base64-encoded ciphertext
784     """
785     ct = self._pub.encrypt(
786         plaintext,
787         _rsa_padding.OAEP( # OAEP is stronger than PKCS#1 v1.5
788             mgf=_rsa_padding.MGF1(algorithm=_hashes.SHA256()),
789             algorithm=_hashes.SHA256(),
790             label=None,
791         )
792     )
793     return base64_encode(ct)
794

```


decoding

```

795     # decryption main section
796     def decrypt(self, ciphertext_b64: str) -> bytes:
797         """
798         Decrypt a Base64-encoded ciphertext with the RSA private key.
799
800         Parameters
801         -----
802         ciphertext_b64 : str – Base64-encoded ciphertext
803
804         Returns
805         -----
806         bytes – original plaintext
807         """
808         if self._key is None:
809             raise RuntimeError("Private key not available – cannot decrypt.")
810         ct = base64_decode(ciphertext_b64)
811         return self._key.decrypt(
812             ct,
813             _rsa_padding.OAEP(
814                 mgf=_rsa_padding.MGF1(algorithm=_hashes.SHA256()),
815                 algorithm=_hashes.SHA256(),
816                 label=None,
817             )
818         )
819

```

Testing Examples

Try Crypto Toolkit

[Home – Passwords Security](#) [Try Crypto Toolkit](#)

Crypto Toolkit

Encrypt, encode, and hash with every algorithm.

Algorithm

- Base64
- Hex
- URL
- SHA-256
- SHA-512
- Salted Hash
- AES
- DES
- 3DES
- RSA-1024
- RSA-2048

Base64

Encoding

Encodes binary data as ASCII text. Not encryption.

Input

YXpteQ==

Encrypt / Encode / Hash

Decrypt / Decode

Output

azmy

Try Crypto Toolkit

[Home](#) – [Passwords](#) [Security](#) **Try Crypto Toolkit**

Crypto Toolkit

Encrypt, encode, and hash with every algorithm.

Algorithm

- Base64**
- Hex
- URL
- SHA-256
- SHA-512
- Salted Hash
- AES
- DES
- 3DES
- RSA-1024
- RSA-2048

Base64 Encoding

Encodes binary data as ASCII text. Not encryption.

Input

YXpteQ==

Encrypt / Encode / Hash

Decrypt / Decode

Output

azmy

Hex

URL

SHA-256

SHA-512

Salted Hash

AES

DES

3DES

RSA-1024

RSA-2048

Converts each byte to two hex digits.

Input

azmy

Encrypt / Encode / Hash

Decrypt / Decode

Output

617a6d79

Hex

URL

SHA-256

SHA-512

Salted Hash

AES

DES

3DES

RSA-1024

RSA-2048

Converts each byte to two hex digits.

Input

617a6d79

Encrypt / Encode / Hash

Decrypt / Decode

Output

azmy

URL	Percent-encodes text for URLs. Supports Arabic and
SHA-256	
SHA-512	
Salted Hash	
AES	
DES	
3DES	
RSA-1024	
RSA-2048	

Input

hello! , azmy ???

Encrypt / Encode / Hash Decrypt / Decode

Output

hello%21%20%2C%20azmy%20%3F%3F%3F

URL	Percent-encodes text for URLs. Supports Arabic and Unicode.
SHA-256	
SHA-512	
Salted Hash	
AES	
DES	
3DES	
RSA-1024	
RSA-2048	

Input

hello%21%20%2C%20azmy%20%3F%3F%3F

Encrypt / Encode / Hash Decrypt / Decode

Output

hello! , azmy ???

SHA-256	
SHA-512	
Salted Hash	
AES	
DES	
3DES	
RSA-1024	
RSA-2048	

Input

azmy

Encrypt / Encode / Hash

Output

cacb288ab9b2959a861220c0abf45abeee9b1cdf668e6a774ff08a330df20b88

SHA-512	Input
Salted Hash	azmy
AES	
DES	
3DES	
RSA-1024	
RSA-2048	
Encrypt / Encode / Hash	
Output	
35b0a6e2ed6554fba205c90475487caa0a30c84da969ee32bcb03237a20a5b073b9d197edb588b5b6ac8e9aa22d6fcae5c4492a3bcb7d6cf93070197b00c71a	

Salted Hash	Input
AES	azmy
DES	
3DES	
RSA-1024	
RSA-2048	
Encrypt / Encode / Hash	
Output	
salt : 671d112e7b508ec9e7e527ae7b596ca2	
hash : ba9d05b4ea1a0882413f68426495a6d8bcf3c151957018284dc21cd623f42c72	

AES

DES

3DES

RSA-1024

RSA-2048

azmy

Encrypt / Encode / Hash

Decrypt / Decode

Output

EdwlrCPdgSWRRc83PLhFQ==

Copy

Key (hex): 95f97b5343666e242e7262af71977f0916 bytes (AES-128)Gen Key

Mode: CBCCBC requires a 16-byte IV (32 hex chars).

IV (16 bytes): b871b157c8873c41cdab9faca59b4aabGen IV

AES

DES

3DES

RSA-1024

RSA-2048

EdwlrCPdgSWRRc83PLhFQ=

Encrypt / Encode / Hash

Decrypt / Decode

Output

azmy

Copy

Key (hex): 95f97b5343666e242e7262af71977f0916 bytes (AES-128)Gen Key

Mode: CBCCBC requires a 16-byte IV (32 hex chars).

IV (16 bytes): b871b157c8873c41cdab9faca59b4aabGen IV

SHA-512
Salted Hash
AES
DES
3DES
RSA-1024
RSA-2048

Input

azmy

Encrypt / Encode / Hash

Decrypt / Decode

Output

VhIlaGDFDc=

Copy

Key (hex, 8 B):

c0a4c11f910198cb

Gen Key

IV (hex, 8 B):

cc7f4603f26e0058

Gen IV

Mode:

CBC

AES
DES
3DES
RSA-1024
RSA-2048

VhIlaGDFDc=

Encrypt / Encode / Hash

Decrypt / Decode

Output

azmy

Copy

Key (hex, 8 B):

c0a4c11f910198cb

Gen Key

IV (hex, 8 B):

cc7f4603f26e0058

Gen IV

Mode:

CBC

SHA-512
Salted Hash
AES
DES
3DES
RSA-1024
RSA-2048

Input

azmy

Encrypt / Encode / Hash

Decrypt / Decode

Output

WmdkaBPUmD4=

Copy

Key (hex):

e8276ead1cd30d977a12ec14c8b03b78

16 bytes (2-key, 112-b)

Gen Key

IV (hex, 8 B):

9cd1c1244209a9db

Gen IV

Mode:

CBC

Salted Hash
AES
DES
3DES
RSA-1024
RSA-2048

WmdkaBPUmD4=

Encrypt / Encode / Hash

Decrypt / Decode

Output

azmy

Copy

Key (hex):

e8276ead1cd30d977a12ec14c8b03b78

16 bytes (2-key, 112-b)

Gen Key

IV (hex, 8 B):

9cd1c1244209a9db

Gen IV

Mode:

CBC

SHA-512 Salted Hash AES DES 3DES RSA-1024 RSA-2048	Input azmy Encrypt / Encode / Hash Decrypt / Decode Output vKYUEi2tox7/vYQgIn8sIieVxeQl/BpKJou/ogQUHxOgmRygvwsF0+ZlshpQgFefdpbzp6neE6HbW2vYs2JKGVfVbVGOEDGVitllVol3IjJnmLqrla78H4gI+RzCr+YBNYh/9bUwktQIfhhdj5QX+1Gk3M01j7k+2f3cT3Q= Copy Keys ready (1024-bit). Copy fields above to save them. Generate Key Pair Public Key (hex): d73342e689315290381e4b9f8cf3f18480427c60a34ee945945dda023fb36e13037c70fb2448df8e5463a7da1103058aa0509e57efafa17f02e13ceebe214092376bac04d5d80cb1fa872a77da6d3d831d98f Private Key (hex): d73342e689315290381e4b9f8cf3f18480427c60a34ee945945dda023fb36e13037c70fb2448df8e5463a7da1103058aa0509e57efafa17f02e13ceebe214092376bac04d5d80ct (needed for decrypt)
---	---

SHA-512 Salted Hash AES DES 3DES RSA-1024 RSA-2048	Input vKYUEi2tox7/vYQgIn8sIieVxeQl/BpKJou/ogQUHxOgmRygvwsF0+ZlshpQgFefdpbzp6neE6HbW2vYs2JKGVfVbVGOEDGVitllVol3IjJnmLqrla78H4gI+RzCr+YBNYh/9bUwktQIfhhdj5QX+1Gk3M01j7k+2f3cT3Q= Encrypt / Encode / Hash Decrypt / Decode Output azmy Copy Keys ready (1024-bit). Copy fields above to save them. Generate Key Pair Public Key (hex): d73342e689315290381e4b9f8cf3f18480427c60a34ee945945dda023fb36e13037c70fb2448df8e5463a7da1103058aa0509e57efafa17f02e13ceebe214092376bac04d5d80cb1fa872a77da6d3d831d98f Private Key (hex): d73342e689315290381e4b9f8cf3f18480427c60a34ee945945dda023fb36e13037c70fb2448df8e5463a7da1103058aa0509e57efafa17f02e13ceebe214092376bac04d5d80ct (needed for decrypt)
---	---

Salted Hash AES DES 3DES RSA-1024 RSA-2048	Input HnxjsjNvVvaGsIj/RIAGoCb0PhtzRgHk3m447F9FR+He6TYU7/V88TOSFPEH2R01RdE8t9PT4T8ESwrf50/nm7Tq2MgyeLh71HF8WcLXYgGB2BKCXnnXV5S5ZhxzGSPsyWryL+HnQg/IJ4x6TP3WQyngEF8EfiBj8Xj8KqPeTCy8aQ7WVZfJdJgJek3pe3btqLPCEm1Gc/b0sDuFpx1AA9B/acifx16t8bEhE8314kX/CYdK5/yocjxhZb153JgOUQ10rY1eJQKmcOQ9KSHmzvZ1YhCzme2QTHvedFx6HrFmmgPyH2zQh3zkpr/1SE16H+HEHESHXvDahQ== Encrypt / Encode / Hash Decrypt / Decode Output azmy Copy Keys ready (2048-bit). Copy fields above to save them. Generate Key Pair Public Key (hex): c1eb209ac1b4753a1415b83ae680be6d5fe720eef1fae758ca4fa2cc507ffc593b7b825dc6d049170e5aa26026bbe5504fca0444fc42655bdcfa1bca73d479752e1de2d859e1bfc35727184e8633e02e09a: Private Key (hex): c1eb209ac1b4753a1415b83ae680be6d5fe720eef1fae758ca4fa2cc507ffc593b7b825dc6d049170e5aa26026bbe5504fca0444fc42655bdcfa1bca73d479752e1de2d859e1bfc (needed for decrypt)
--	--

Salted Hash AES DES 3DES RSA-1024 RSA-2048	Input HnxjsjNvVvaGsIj/RIAGoCb0PhtzRgHk3m447F9FR+He6TYU7/V88TOSFPEH2R01RdE8t9PT4T8ESwrf50/nm7Tq2MgyeLh71HF8WcLXYgGB2BKCXnnXV5S5ZhxzGSPsyWryL+HnQg/IJ4x6TP3WQyngEF8EfiBj8Xj8KqPeTCy8aQ7WVZfJdJgJek3pe3btqLPCEm1Gc/b0sDuFpx1AA9B/acifx16t8bEhE8314kX/CYdK5/yocjxhZb153JgOUQ10rY1eJQKmcOQ9KSHmzvZ1YhCzme2QTHvedFx6HrFmmgPyH2zQh3zkpr/1SE16H+HEHESHXvDahQ== Encrypt / Encode / Hash Decrypt / Decode Output azmy Copy Keys ready (2048-bit). Copy fields above to save them. Generate Key Pair Public Key (hex): c1eb209ac1b4753a1415b83ae680be6d5fe720eef1fae758ca4fa2cc507ffc593b7b825dc6d049170e5aa26026bbe5504fca0444fc42655bdcfa1bca73d479752e1de2d859e1bfc35727184e8633e02e09a: Private Key (hex): c1eb209ac1b4753a1415b83ae680be6d5fe720eef1fae758ca4fa2cc507ffc593b7b825dc6d049170e5aa26026bbe5504fca0444fc42655bdcfa1bca73d479752e1de2d859e1bfc (needed for decrypt)
--	--